

Solution Requirements

Date	06-07-2026
Team ID	
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

This document lists the requirements verified against the actual implemented OptiCrop application — the Flask backend, machine learning crop-prediction pipeline, SQLite data layer, prediction history, dashboard, CSV export, and PDF report generation — rather than a planned or aspirational feature set.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should *do*).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should *be*).

Where a template category is not implemented in the project (e.g., authentication, external APIs, compliance standards), this is stated accurately rather than invented.

Step 1: Functional Requirements (FR)

S. No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	No login or credential-based authentication is implemented. Every request is automatically attributed to a single auto-created “guest” user record so predictions can be logged; a password_hash field exists in the schema but is unused.	Low
2	Authorization levels	No role-based access control exists. The single shared guest account has identical, unrestricted access to prediction, history, dashboard, CSV export, and PDF report features — no admin/user role distinction is implemented.	Low
3	External interfaces	No third-party APIs or external databases are called. The Flask app exposes its own JSON endpoints (/api/predict, /api/history, /api/health, /api/delete) used by the bundled HTML pages and an optional local React dev frontend (CORS-enabled).	Medium

4	Transactions processing	No payment/financial transactions occur. The core transaction is a prediction request: validated N, P, K, temperature, humidity, pH and rainfall values are scaled, passed to the trained model, and the result is committed as one Prediction record in SQLite.	High
5	Reporting	The system provides a prediction history table with CSV export, a per-prediction PDF report (FPDF), and a dashboard showing recent predictions, crop-distribution counts, and per-model accuracy/precision/recall/F1 comparison metrics.	High
6	Business rules, etc.	Submitted soil/climate values must be numeric and non-negative or the request is rejected with a flash message. Crop season, water need, and fertilizer guidance are derived from a static reference lookup keyed by the predicted crop label.	Medium
7	Compliance to laws or regulations	No specific legal, industry, or data-privacy standard (e.g., GDPR, HIPAA) is implemented or referenced in the code. Only agronomic input values and a guest user record are stored; no sensitive personal data is collected.	Low
8	Other	Additional implemented features: deleting a prediction record (DELETE /api/delete/<id>), a health-check endpoint (/api/health), static About/Contact pages, and a custom 404 error page.	Medium

Step 2: Non-Functional Requirements (NFR)

S. No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Predictions run synchronously in-process using a scikit-learn model loaded once at startup, returning a result within the same request/response cycle without background/async processing.	No specific response-time target is defined or benchmarked in the project.
2	Scalability	The app runs as a single Flask process (optionally under Gunicorn per the included Procfile) backed by a local SQLite database — suited to single-instance use rather than distributed scaling.	No concurrent-user or load target is defined or tested in the project.
3	Security & Data Privacy	No authentication, authorization, or encryption-at-rest is implemented. A Flask secret_key (from environment variable, with a development fallback) is used only for session/flash-message signing.	No specific encryption standard or vulnerability-management target is defined.
4	Reliability & Availability	A custom 404 handler and try/except error handling around the delete endpoint are implemented; input validation blocks invalid predictions. No automated failover, retries, or uptime monitoring exist.	No uptime percentage target is defined or measured.

5	Usability & Accessibility	Server-rendered HTML pages (Home, Result, History, Dashboard, About, Contact) with flash messages give users clear feedback on invalid input during prediction submission.	No formal accessibility standard (e.g., WCAG) is implemented or tested.
6	Other	Maintainability is supported via separation of concerns across app.py (routes), model.py (ML pipeline), and database.py (data layer), plus a train_model.py script to retrain artifacts from the bundled dataset.	Not applicable / not measured.